

UNIVERSITY OF INFORMATION TECHNOLOGY, VNUHCM

FACULTY OF INFORMATION SYSTEM

SEARCHING AND SORTING ALGORITHMS

Laboratory: **LAB01**

Teacher: **Võ Minh Tâm**

Name	ID
Trần Anh Tuấn	24521943

OTHER EVALUATIONS (*):

Content	Result
Average total time to complete the lab ⁽¹⁾	5 hours
Video Link ⁽²⁾ (if any)	
Feedback ⁽³⁾ (if any) + Challenges + Suggestions ...	
Self-Assessment Score ⁽⁴⁾	10/10

TASK ASSIGNMENT TABLE:

Members	Task Details
Member 1	Task 1 (Work - 100%)

Table of Contents

1.1 Linear Search	1
1.2 Binary Search	2
1.3 Selection Sort	3
1.4 Insertion Sort	5
1.5 Bubble Sort	6
1.6 Quick Sort	7
1.7 Heap Sort	9
1.8 Radix Sort	10
2.1 Jump Search	12
2.2 Ternary Search	13
2.3 Interpolation Search	15
2.4 Exponential Search	17
2.5 Counting Sort	18
2.6 Bucket Sort	20
2.7 Merge sort	22
2.8 Tim Sort	23

1.1 Linear Search

Algorithm Idea

The algorithm goes through the array one element at a time, from left to right. At each index, it checks whether the current element is equal to the search value (target). If a match is found, the index is returned immediately. If the end of the array is reached without a match, the function returns -1.

Characteristics

- **Best Case:** $O(1)$, when the target is the first element.
- **Worst Case:** $O(n)$, when the target is the last element or not present.
- **Average Case:** $O(n)$.
- **Space Complexity:** $O(1)$, as no extra memory is needed.

Codes

```
#include <bits/stdc++.h>

using namespace std;

int linearSearch(int arr[], int n, int target) {
    // input: array, array size, search value

    for (int i = 0; i < n; i++){ // loop through each element
        if (arr[i] == target){ // check if current element matches target
            return i; // return index if found
        }
    }
    return -1 // return -1 if not found

    // output: index if found, otherwise -1
}

int main() {
    int n = 6;
    int arr[n] = {-14, -5, 1, 3, -17, 2};
    int target;

    target = 3;
    cout << linearSearch(arr, n, target) << '\n'; // Output: 3

    return 0;
}
```

Visualization

- Example

Array: [-14, -5, 1, 3, -17, 2]

Target: 3

- **Steps**

1. Compare `arr[0] = -14` with `3` → not equal
2. Compare `arr[1] = -5` with `3` → not equal
3. Compare `arr[2] = 1` with `3` → not equal
4. Compare `arr[3] = 3` with `3` → **match found** → return index `3`

- **Result**

The function returns `3`.

1.2 Binary Search

Algorithm Idea

Binary Search works on a **sorted array**. It repeatedly divides the search range in half by comparing the middle element with the target.

- If the middle element equals the target → return its index.
 - If the target is smaller → continue searching the left half.
 - If the target is larger → continue searching the right half.
- If the search range becomes empty, the function returns `-1`.

Characteristics

- **Best Case:** $O(1)$, when the middle element is the target.
- **Worst Case:** $O(\log n)$, when we keep halving until the element is found or the range is empty.
- **Average Case:** $O(\log n)$.
- **Space Complexity:** $O(1)$, iterative version.
- **Requirement:** Input array must be sorted.

Codes:

```
#include <bits/stdc++.h>
using namespace std;

int binarySearch(int arr[], int n, int target) {
    // input: sorted array, array size, search value
    int left = 0, right = n - 1;

    while (left <= right) { // search until range is valid
        int mid = (left + right) / 2; // middle index
```

```

        if (arr[mid] == target)           // found target
            return mid;
        else if (arr[mid] < target)       // target is in right half
            left = mid + 1;
        else                             // target is in left half
            right = mid - 1;
    }
    return -1; // not found
    // output: index if found, otherwise -1
}

int main() {
    int n = 6;
    int arr[n] = {-17, -14, -5, 1, 2, 3}; // sorted array
    int target;

    target = 1;
    cout << binarySearch(arr, n, target) << '\n'; // Output: 3

    return 0;
}

```

Visualization

- **Example**

Array: [-17, -14, -5, 1, 2, 3]

Target: 1

- **Steps**

1. left = 0, right = 5 → mid = 2 → arr[2] = -5 → target (1) > -5 → search right half.
2. left = 3, right = 5 → mid = 4 → arr[4] = 2 → target (1) < 2 → search left half.
3. left = 3, right = 3 → mid = 3 → arr[3] = 1 → **match found** → return index 3.

- **Result**

The function returns 3.

1.3 Selection Sort

Algorithm Idea

The algorithm divides the array into two parts: a sorted part and an unsorted part. On each pass, it selects the smallest element from the unsorted part and swaps it with the first unsorted element, thereby expanding the sorted part by one.

Characteristics

- **Best Case:** $O(n^2)$
- **Worst Case:** $O(n^2)$

- Average Case: $O(n^2)$
- Space Complexity: $O(1)$

Codes

```
#include <bits/stdc++.h>
using namespace std;

void selectionSort(int arr[], int n) {
    // input: array and its size

    for (int i = 0; i < n - 1; i++) { // move boundary of unsorted part
        int minIdx = i; // assume current element is the minimum
        for (int j = i + 1; j < n; j++) { // find the smallest element in
            unsorted part
                if (arr[j] < arr[minIdx])
                    minIdx = j; // update minIdx if smaller element found
        }
        swap(arr[i], arr[minIdx]); // place minimum at the beginning of
        unsorted part
    }
    // output: sorted array
}

int main() {
    int n = 5;
    int arr[n] = {5, 3, 4, 1, 2}; // input array

    selectionSort(arr, n); // sort the array

    for (int i = 0; i < n; i++)
        cout << arr[i] << " "; // expected output: 1 2 3 4 5
    return 0;
}
```

Visualization

- Example:

Array: [5, 3, 4, 1, 2]

- Steps
 1. Pass 1: Smallest = 1 → swap with 5 → [1, 3, 4, 5, 2]
 2. Pass 2: Smallest = 2 → swap with 3 → [1, 2, 4, 5, 3]
 3. Pass 3: Smallest = 3 → swap with 4 → [1, 2, 3, 5, 4]
 4. Pass 4: Smallest = 4 → swap with 5 → [1, 2, 3, 4, 5]
- Result:

The function returns the sorted array: [1, 2, 3, 4, 5]

1.4 Insertion Sort

Algorithm Idea

The algorithm builds the sorted part of the array one element at a time. Each new element is compared with the sorted elements and inserted into its correct position by shifting larger elements to the right.

Characteristics

- **Best Case:** $O(n)$, when the array is already sorted.
- **Worst Case:** $O(n^2)$, when the array is sorted in reverse.
- **Average Case:** $O(n^2)$.
- **Space Complexity:** $O(1)$.

Codes

```
#include <bits/stdc++.h>
using namespace std;

void insertionSort(int arr[], int n) {
    // input: array and its size

    for (int i = 1; i < n; i++) { // loop through elements starting from index
1
        int key = arr[i]; // store current element
        int j = i - 1;
        while (j >= 0 && arr[j] > key) { // shift elements greater than key
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key; // insert key at the correct position
    }
    // output: sorted array
}

int main() {
    int n = 5;
    int arr[n] = {5, 3, 4, 1, 2}; // input array

    insertionSort(arr, n);

    for (int i = 0; i < n; i++)
        cout << arr[i] << " "; // expected output: 1 2 3 4 5
    return 0;
}
```

Visualization

- **Example**

Array: [5, 3, 4, 1, 2]

- **Steps**

1. Insert 3 into [5] → [3, 5, 4, 1, 2]
2. Insert 4 into [3, 5] → [3, 4, 5, 1, 2]
3. Insert 1 into [3, 4, 5] → [1, 3, 4, 5, 2]
4. Insert 2 into [1, 3, 4, 5] → [1, 2, 3, 4, 5]

- **Result**

The function returns the sorted array: [1, 2, 3, 4, 5]

1.5 Bubble Sort

Algorithm Idea

The algorithm repeatedly compares adjacent elements and swaps them if they are in the wrong order. After each pass, the largest unsorted element bubbles up to its correct position at the end.

Characteristics

- **Best Case:** $O(n)$, when the array is already sorted.
- **Worst Case:** $O(n^2)$.
- **Average Case:** $O(n^2)$.
- **Space Complexity:** $O(1)$.

Codes

```
#include <bits/stdc++.h>
using namespace std;

void bubbleSort(int arr[], int n) {
    // input: array and its size

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            // loop for each pass
            // compare adjacent
            // swap if out of order
            if (arr[j] > arr[j + 1])
                swap(arr[j], arr[j + 1]);
        }
    }
}
```

```

    // output: sorted array
}

int main() {
    int n = 5;
    int arr[n] = {5, 3, 4, 1, 2}; // input array

    bubbleSort(arr, n);

    for (int i = 0; i < n; i++)
        cout << arr[i] << " "; // expected output: 1 2 3 4 5
    return 0;
}

```

Visualization

- **Example**

Array: [5, 3, 4, 1, 2]

- **Steps**
 1. Pass 1: [3, 4, 1, 2, 5]
 2. Pass 2: [3, 1, 2, 4, 5]
 3. Pass 3: [1, 2, 3, 4, 5]
 4. Pass 4: [1, 2, 3, 4, 5]
- **Result**

The function returns the sorted array: [1, 2, 3, 4, 5]

1.6 Quick Sort

Algorithm Idea

Quick Sort chooses a **pivot element** and partitions the array so that all elements smaller than the pivot are placed on the left and all larger elements are placed on the right. It then recursively sorts the left and right subarrays.

Characteristics

- **Best Case:** $O(n \log n)$, when partitions are balanced.
- **Worst Case:** $O(n^2)$, when partitions are unbalanced
- **Average Case:** $O(n \log n)$.

- **Space Complexity:** $O(\log n)$, due to recursion.

Codes

```
#include <bits/stdc++.h>
using namespace std;

int partition(int arr[], int low, int high) {
    int pivot = arr[high]; // choose last element as pivot
    int i = low - 1; // index of smaller element
    for (int j = low; j < high; j++) { // traverse array
        if (arr[j] <= pivot) { // if current element <= pivot
            i++;
            swap(arr[i], arr[j]); // place smaller element on left
        }
    }
    swap(arr[i + 1], arr[high]); // place pivot in correct position
    return i + 1; // return pivot index
}

void quickSort(int arr[], int low, int high) {
    // input: array, left index, right index
    if (low < high) {
        int pi = partition(arr, low, high); // partition index
        quickSort(arr, low, pi - 1); // recursively sort left
        quickSort(arr, pi + 1, high); // recursively sort right
    }
    // output: sorted array
}

int main() {
    int n = 5;
    int arr[n] = {5, 3, 4, 1, 2}; // input array

    quickSort(arr, 0, n - 1);

    for (int i = 0; i < n; i++)
        cout << arr[i] << " "; // expected output: 1 2 3 4 5
    return 0;
}
```

Visualization

- Example

Array: [5, 3, 4, 1, 2]

- Steps

1. Pivot = 2 → partition → [1, 2, 4, 5, 3]
2. Left subarray [1] (already sorted)
3. Right subarray [4, 5, 3], Pivot = 3 → partition → [3, 4, 5]

4. Final result: [1, 2, 3, 4, 5]

- **Result**

The function returns the sorted array: [1, 2, 3, 4, 5]

1.7 Heap Sort

Algorithm Idea

Heap Sort builds a **max heap** from the array. The largest element (at the root) is swapped with the last element. The heap size is reduced, and the heap is rebuilt. This process repeats until the array is sorted.

Characteristics

- **Best Case:** $O(n \log n)$.
- **Worst Case:** $O(n \log n)$.
- **Average Case:** $O(n \log n)$.
- **Space Complexity:** $O(1)$.

Codes

```
#include <bits/stdc++.h>
using namespace std;
```

```
void heapify(int arr[], int n, int i) {
    int largest = i; // assume root is largest
    int l = 2 * i + 1; // left child
    int r = 2 * i + 2; // right child

    if (l < n && arr[l] > arr[largest]) // if left > root
        largest = l;
    if (r < n && arr[r] > arr[largest]) // if right > largest
        largest = r;

    if (largest != i) { // if largest not root
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest); // recursively heapify
    }
}
```

```
void heapSort(int arr[], int n) {
    // input: array and its size
    for (int i = n / 2 - 1; i >= 0; i--) // build max heap
        heapify(arr, n, i);
    for (int i = n - 1; i > 0; i--) { // extract elements
        swap(arr[0], arr[i]); // move max to end
```

```

        heapify(arr, i, 0); // heapify reduced heap
    }
    // output: sorted array
}

int main() {
    int n = 5;
    int arr[n] = {5, 3, 4, 1, 2}; // input array

    heapSort(arr, n);

    for (int i = 0; i < n; i++)
        cout << arr[i] << " "; // expected output: 1 2 3 4 5
    return 0;
}

```

Visualization

- **Example**

Array: [5, 3, 4, 1, 2]

- **Steps**

1. Build max heap $\rightarrow$ [5, 3, 4, 1, 2]
2. Swap 5 with last $\rightarrow$ [2, 3, 4, 1, 5], heapify $\rightarrow$ [4, 3, 2, 1, 5]
3. Swap 4 with last unsorted $\rightarrow$ [1, 3, 2, 4, 5], heapify $\rightarrow$ [3, 1, 2, 4, 5]
4. Swap 3 with last unsorted $\rightarrow$ [2, 1, 3, 4, 5], heapify $\rightarrow$ [2, 1, 3, 4, 5]
5. Swap 2 with last unsorted $\rightarrow$ [1, 2, 3, 4, 5]

- **Result**

The function returns the sorted array: [1, 2, 3, 4, 5]

1.8 Radix Sort

Algorithm Idea

Radix Sort processes the digits of numbers one by one, starting from the least significant digit to the most significant digit. At each step, a **stable counting sort** is applied on the current digit.

Characteristics

- **Best Case:** $O(d \cdot (n+k))$
- **Worst Case:** $O(d \cdot (n+k))$
- **Average Case:** $O(d \cdot (n+k))$, where d = number of digits, k = range of digits

- Space Complexity: $O(n + k)$.
- Works only for integers.

Codes

```
#include <bits/stdc++.h>
using namespace std;

int getMax(int arr[], int n) {
    int mx = arr[0]; // assume first as max
    for (int i = 1; i < n; i++) // find maximum
        if (arr[i] > mx) mx = arr[i];
    return mx;
}

void countingSort(int arr[], int n, int exp) {
    int output[n]; // output array
    int count[10] = {0}; // count array for digits 0-9

    for (int i = 0; i < n; i++) // count occurrences
        count[(arr[i] / exp) % 10]++;

    for (int i = 1; i < 10; i++) // cumulative count
        count[i] += count[i - 1];

    for (int i = n - 1; i >= 0; i--) { // build output (stable)
        output[count[(arr[i] / exp) % 10] - 1] = arr[i];
        count[(arr[i] / exp) % 10]--;
    }

    for (int i = 0; i < n; i++) // copy back
        arr[i] = output[i];
}

void radixSort(int arr[], int n) {
    // input: array and its size
    int m = getMax(arr, n); // get max to know digit count
    for (int exp = 1; m / exp > 0; exp *= 10) // sort by each digit
        countingSort(arr, n, exp);
    // output: sorted array
}

int main() {
    int n = 5;
    int arr[n] = {170, 45, 75, 90, 802}; // input array

    radixSort(arr, n);

    for (int i = 0; i < n; i++)
        cout << arr[i] << " "; // expected output: 45 75 90 170 802
    return 0;
}
```

Visualization

- **Example**

Array: [170, 45, 75, 90, 802]

- **Steps**

1. Sort by 1s digit → [170, 90, 802, 45, 75]
2. Sort by 10s digit → [802, 45, 75, 170, 90]
3. Sort by 100s digit → [45, 75, 90, 170, 802]

- **Result**

The function returns the sorted array: [45, 75, 90, 170, 802]

2.1 Jump Search

Algorithm Idea

Jump Search works on a **sorted array**. Instead of checking every element, it jumps ahead by fixed steps (usually $\sqrt{n}$). When the range containing the target is found, it does a linear search inside that block.

Characteristics

- **Best Case:** $O(1)$
- **Worst Case:** $O(\sqrt{n})$
- **Average Case:** $O(\sqrt{n})$
- **Space Complexity:** $O(1)$
- **Requires sorted input.**

Codes

```
#include <bits/stdc++.h>
using namespace std;

int jumpSearch(int arr[], int n, int target) {
    // input: sorted array, array size, search value

    int step = sqrt(n); // block size
    int prev = 0;

    while (arr[min(step, n) - 1] < target) { // jump until value >= target
        prev = step;
```

```

        step += sqrt(n);
        if (prev >= n) return -1; // not found
    }

    for (int i = prev; i < min(step, n); i++) // linear search inside block
        if (arr[i] == target)
            return i; // return index if found

    return -1; // return -1 if not found

    // output: index if found, otherwise -1
}

int main() {
    int n = 7;
    int arr[n] = {1, 3, 5, 7, 9, 11, 13};
    int target = 7;
    cout << jumpSearch(arr, n, target) << '\n'; // Output: 3
    return 0;
}

```

Visualization

- **Example**

Array: [1, 3, 5, 7, 9, 11, 13]

Target: 7

- **Steps**

1. Step size = $\sqrt{7} \approx 2 \rightarrow$ Compare $\text{arr}[1] = 3$ with 7 $\rightarrow 3 < 7 \rightarrow$ keep jumping.
2. Jump to next block $\rightarrow$ step = 4 $\rightarrow$ Compare $\text{arr}[3] = 7$ with 7 $\rightarrow$ **match found** $\rightarrow$ return index 3.

- **Result**

The function returns 3.

2.2 Ternary Search

Algorithm Idea

Ternary Search is a **divide and conquer** method that works on a sorted array. It splits the array into three parts by finding two midpoints. The target is compared with the values at these midpoints to decide which segment to continue searching.

Characteristics

- **Best Case:** $O(1)$, when the target is at a midpoint.
- **Worst Case:** $O(\log_3 n)$.
- **Average Case:** $O(\log_3 n)$.
- **Space Complexity:** $O(1)$ (iterative).
- **Requirement:** The array must be sorted.

Codes

```
#include <bits/stdc++.h>
using namespace std;

int ternarySearch(int arr[], int l, int r, int target) {
    // input: sorted array, left index, right index, search value

    while (r >= l) {
        int mid1 = l + (r - l) / 3; // first mid
        int mid2 = r - (r - l) / 3; // second mid

        if (arr[mid1] == target) return mid1; // return index if found at
mid1
        if (arr[mid2] == target) return mid2; // return index if found at
mid2

        if (target < arr[mid1]) r = mid1 - 1; // search left part
        else if (target > arr[mid2]) l = mid2 + 1; // search right part
        else { l = mid1 + 1; r = mid2 - 1; } // search middle part
    }

    return -1; // return -1 if not found

    // output: index if found, otherwise -1
}

int main() {
    int n = 6;
    int arr[n] = {1, 3, 5, 7, 9, 11};
    int target = 7;
```

```

        cout << ternarySearch(arr, 0, n - 1, target) << '\n'; // Output: 3
    return 0;
}

```

Visualization

- **Example**

Array: [1, 3, 5, 7, 9, 11]

Target: 7

- **Steps**

1. $L = 0, R = 5 \rightarrow \text{mid1} = 1$ ($\text{arr}[1] = 3$), $\text{mid2} = 4$ ($\text{arr}[4] = 9$). Target 7 lies between them $\rightarrow$ search middle segment.
2. New range $L = 2, R = 3 \rightarrow \text{mid1} = 2$ ($\text{arr}[2] = 5$), $\text{mid2} = 3$ ($\text{arr}[3] = 7$).
3. Compare $\text{arr}[3] = 7$ with target $\rightarrow$ **match found** $\rightarrow$ return index 3.

- **Result**

The function returns 3.

2.3 Interpolation Search

Algorithm Idea

Interpolation Search improves on Binary Search by estimating the position of the target using interpolation (like searching in a phonebook). It works best when the array is sorted and values are **uniformly distributed**.

Characteristics

- **Best Case:** $O(1)$.
- **Worst Case:** $O(n)$, when distribution is skewed.
- **Average Case:** $O(\log \log n)$.
- **Space Complexity:** $O(1)$.

- **Requirement:** The array must be sorted with near-uniform distribution.

Codes

```
#include <bits/stdc++.h>
using namespace std;

int interpolationSearch(int arr[], int n, int target) {
    // input: sorted array, array size, search value
    int low = 0, high = n - 1;
    while (low <= high && target >= arr[low] && target <= arr[high]) {
        if (low == high) {
            if (arr[low] == target) return low;
            return -1;
        }
        // estimate position
        int pos = low + ((double)(high - low) / (arr[high] - arr[low])) *
(target - arr[low]);
        if (arr[pos] == target) return pos; // return index if found
        if (arr[pos] < target) low = pos + 1; // search right
        else high = pos - 1; // search left
    }
    return -1; // return -1 if not found
    // output: index if found, otherwise -1
}

int main() {
    int n = 5;
    int arr[n] = {10, 20, 30, 40, 50};
    int target = 30;
    cout << interpolationSearch(arr, n, target) << '\n'; // Output: 2
    return 0;
}
```

Visualization

- **Example**

Array: [10, 20, 30, 40, 50]

Target: 30

- **Steps**

1. Low = 0, high = 4 → Estimate pos = $0 + ((30 - 10) * (4 - 0)) / (50 - 10) = 2$.
2. Compare arr[2] = 30 with target → **match found** → return index 2.

- **Result**

The function returns 2.

2.4 Exponential Search

Algorithm Idea

Exponential Search first finds a range where the element may exist by repeatedly doubling the index. Once the range is found, it applies Binary Search within that range.

Characteristics

- **Best Case:** $O(1)$.
- **Worst Case:** $O(\log n)$.
- **Average Case:** $O(\log n)$.
- **Space Complexity:** $O(1)$.
- **Requirement:** The array must be sorted.

Codes

```
#include <bits/stdc++.h>
using namespace std;

int binarySearch(int arr[], int left, int right, int target) {
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) return mid; // return index if found
        if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1; // not found
}

int exponentialSearch(int arr[], int n, int target) {
    // input: sorted array, array size, search value

    if (arr[0] == target) return 0;

    int i = 1;
    while (i < n && arr[i] <= target)
        i *= 2; // find range

    return binarySearch(arr, i/2, min(i, n-1), target); // search in range

    // output: index if found, otherwise -1
}
```

```
}
```

```
int main() {  
    int n = 7;  
    int arr[n] = {1, 2, 3, 4, 5, 6, 7};  
    int target = 6;  
  
    cout << exponentialSearch(arr, n, target) << '\n'; // Output: 5  
    return 0;  
}
```

Visualization

- **Example**

Array: [1, 2, 3, 4, 5, 6, 7]

Target: 6

- **Steps**

1. Compare arr[1] = 2 with 6 → less → double index.
2. Compare arr[2] = 3 with 6 → less → double index.
3. Compare arr[4] = 5 with 6 → less → double index.
4. Range found between index 4 and 6.
5. Apply Binary Search in [5,6,7] → mid = 5 → arr[5] = 6 → **match found** → return 5.

- **Result**

The function returns 5.

2.5 Counting Sort

Algorithm Idea

Counting Sort counts the frequency of each element, calculates the cumulative count, and places each element in its correct position in the output array. It works only for integers in a limited range.

Characteristics

- Best Case: $O(n + k)$
- Worst Case: $O(n + k)$
- Average Case: $O(n + k)$
- Space Complexity: $O(n + k)$

Codes

```
#include <bits/stdc++.h>
using namespace std;

void countingSort(int arr[], int n) {
    // input: array and its size

    int maxVal = *max_element(arr, arr + n); // find maximum value
    int count[maxVal + 1] = {0}; // count array

    for (int i = 0; i < n; i++) // count each element
        count[arr[i]]++;

    for (int i = 1; i <= maxVal; i++) // cumulative sum
        count[i] += count[i - 1];

    int output[n]; // output array
    for (int i = n - 1; i >= 0; i--) { // build output (stable)
        output[count[arr[i]] - 1] = arr[i];
        count[arr[i]]--;
    }

    for (int i = 0; i < n; i++) // copy back
        arr[i] = output[i];

    // output: sorted array
}

int main() {
    int n = 7;
    int arr[n] = {4, 2, 2, 8, 3, 3, 1};
    countingSort(arr, n);

    for (int i = 0; i < n; i++)
        cout << arr[i] << " "; // Output: 1 2 2 3 3 4 8
    return 0;
}
```

Visualization

- **Example**

Array: [4, 2, 2, 8, 3, 3, 1]

- **Steps**

1. Count frequencies $\rightarrow [0, 1, 2, 2, 1, 0, 0, 1]$
2. Cumulative count $\rightarrow [0, 1, 3, 5, 6, 6, 6, 7]$
3. Place elements into output array $\rightarrow [1, 2, 2, 3, 3, 4, 8]$

- **Result**

The function returns the sorted array: [1, 2, 2, 3, 3, 4, 8]

2.6 Bucket Sort

Algorithm Idea

Bucket Sort distributes elements into several buckets, sorts each bucket (commonly using Insertion Sort), and then concatenates all buckets to get the final sorted array.

Characteristics

- **Best Case:** $O(n + k)$
- **Worst Case:** $O(n^2)$ (all elements in one bucket)
- **Average Case:** $O(n + k)$
- **Space Complexity:** $O(n + k)$

Codes

```
#include <bits/stdc++.h>
using namespace std;

void bucketSort(float arr[], int n) {
    // input: array and its size
```

```

vector<float> buckets[n]; // create n buckets

for (int i = 0; i < n; i++) { // put elements in buckets
    int idx = n * arr[i]; // index for bucket
    buckets[idx].push_back(arr[i]);
}

for (int i = 0; i < n; i++) // sort each bucket
    sort(buckets[i].begin(), buckets[i].end());

int index = 0; // merge buckets
for (int i = 0; i < n; i++) {
    for (float x : buckets[i])
        arr[index++] = x;
}

// output: sorted array
}

int main() {
    int n = 5;
    float arr[n] = {0.78, 0.17, 0.39, 0.26, 0.72};
    bucketSort(arr, n);

    for (int i = 0; i < n; i++)
        cout << arr[i] << " "; // Output: 0.17 0.26 0.39 0.72 0.78
    return 0;
}

```

Visualization

- Example

Array: [0.78, 0.17, 0.39, 0.26, 0.72]

- Steps

1. Distribute into buckets:

- Bucket0: [0.17]
- Bucket1: [0.26, 0.39]
- Bucket3: [0.72, 0.78]

2. Sort each bucket → [0.17], [0.26, 0.39], [0.72, 0.78]

3. Concatenate → [0.17, 0.26, 0.39, 0.72, 0.78]

- **Result**

The function returns the sorted array: [0.17, 0.26, 0.39, 0.72, 0.78]

2.7 Merge sort

Algorithm Idea

Merge Sort is a **divide and conquer** algorithm. It splits the array into halves, recursively sorts them, and then merges the sorted halves into one.

Characteristics

- **Best Case:** $O(n \log n)$
- **Worst Case:** $O(n \log n)$
- **Average Case:** $O(n \log n)$
- **Space Complexity:** $O(n)$

Codes

```
#include <bits/stdc++.h>
using namespace std;

void merge(int arr[], int l, int m, int r) {
    int n1 = m - l + 1, n2 = r - m; // sizes
    int L[n1], R[n2];

    for (int i = 0; i < n1; i++) L[i] = arr[l + i]; // copy left
    for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j]; // copy right

    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2) { // merge
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++]; // copy leftovers
    while (j < n2) arr[k++] = R[j++];
}

void mergeSort(int arr[], int l, int r) {
    // input: array, left index, right index
    if (l < r) {
        int m = (l + r) / 2;
```

```

        mergeSort(arr, l, m); // sort left half
        mergeSort(arr, m + 1, r); // sort right half
        merge(arr, l, m, r); // merge halves
    }
    // output: sorted array
}

int main() {
    int n = 5;
    int arr[n] = {5, 3, 4, 1, 2};
    mergeSort(arr, 0, n - 1);

    for (int i = 0; i < n; i++)
        cout << arr[i] << " "; // Output: 1 2 3 4 5
    return 0;
}

```

Visualization

- **Example**

Array: [5, 3, 4, 1, 2]

- **Steps**

1. Split into [5,3,4] and [1,2].
2. Split [5,3,4] → [5,3] and [4].
3. Sort [5,3] → [3,5]. Merge with [4] → [3,4,5].
4. Sort [1,2] → already sorted.
5. Merge [3,4,5] and [1,2] → [1,2,3,4,5].

- **Result**

The function returns the sorted array: [1, 2, 3, 4, 5]

2.8 Tim Sort

Algorithm Idea

Tim Sort is a **hybrid sorting algorithm** (used in Python and Java). It uses Insertion Sort for small chunks (runs), and Merge Sort to merge those runs. It is efficient for real-world data which is often partially sorted.

Characteristics

- **Best Case:** $O(n)$ (already sorted)
- **Worst Case:** $O(n \log n)$
- **Average Case:** $O(n \log n)$
- **Space Complexity:** $O(n)$

Codes

```
#include <bits/stdc++.h>
using namespace std;
const int RUN = 32;

void insertionSort(int arr[], int left, int right) {
    // input: array, left and right bounds of subarray

    for (int i = left + 1; i <= right; i++) {
        int key = arr[i]; // take current element
        int j = i - 1;

        // shift elements larger than key
        while (j >= left && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key; // insert key at correct place
    }
    // output: sorted subarray
}

void merge(int arr[], int l, int m, int r) {
    int n1 = m - l + 1, n2 = r - m; // sizes of subarrays
    int L[n1], R[n2]; // temp arrays

    for (int i = 0; i < n1; i++) L[i] = arr[l + i]; // copy left half

    for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j]; // copy right half

    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2) { // merge step
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
}
```

```

    }

    while (i < n1) arr[k++] = L[i++]; // copy remaining L
    while (j < n2) arr[k++] = R[j++]; // copy remaining R
}

void timSort(int arr[], int n) {
    // input: array and its size

    // sort small subarrays using Insertion Sort
    for (int i = 0; i < n; i += RUN)
        insertionSort(arr, i, min(i + RUN - 1, n - 1));

    // merge sorted runs using Merge Sort
    for (int size = RUN; size < n; size = 2 * size) {
        for (int left = 0; left < n; left += 2 * size) {
            int mid = min(left + size - 1, n - 1);
            int right = min(left + 2 * size - 1, n - 1);

            if (mid < right) merge(arr, left, mid, right);
        }
    }
    // output: sorted array
}

int main() {
    int n = 5;
    int arr[n] = {5, 21, 7, 23, 19};

    timSort(arr, n);

    for (int i = 0; i < n; i++)
        cout << arr[i] << " "; // expected output: 5 7 19 21 23
    return 0;
}

```

Visualization

- **Example**

Array: [5, 21, 7, 23, 19]

- **Steps**

1. Break into runs of size $\leq 32 \rightarrow [5, 21], [7, 23], [19]$.
2. Sort each run with Insertion Sort $\rightarrow [5, 21], [7, 23], [19]$.
3. Merge runs $\rightarrow$ merge $[5, 21]$ and $[7, 23] \rightarrow [5, 7, 21, 23]$.

4. Merge with [19] $\rightarrow$ [5, 7, 19, 21, 23].

- **Result**

The function returns the sorted array: [5, 7, 19, 21, 23]

END